BARDWS

Driver Earnings Data Pipeline

Product Requirements Document / Your earnings, your data

Context

The Bengaluru Auto-Rickshaw Drivers' Welfare Sangha (BARDWS) is a Karnataka Society representing about 4,200 auto-rickshaw drivers. It holds years of earnings, fuel, and weather data in mixed formats and wants a documented data foundation before any dashboard or model is built. This milestone delivers only that foundation: parse the four aggregator earnings PDF layouts into one normalized table, tidy and enrich the driver-log sample, and write one documented, de-identified dataset, plus a short validation notebook.

This milestone is a data deliverable, not an app, a model, or a deployed product. There is no dashboard, no forecasting, no encryption layer, and no localization. Those return in later milestones and are listed under Scope Boundaries.

Project Information

Field	Details
Project Type	Multi-source parsing, cleaning, and joining into a documented dataset
Client / Brand	Bengaluru Auto-Rickshaw Drivers' Welfare Sangha (BARDWS)
Platform / Medium	Python 3.11+ command-line pipeline; no UI
Deliverable Type	Parser .py plus datasets .parquet and .csv plus notebook .ipynb plus docs .md
Primary Goal	Turn the provided sample sources into one cleaned, de-identified, documented dataset, reproducibly

Main Goal

Deliver one reproducible Python pipeline that parses all four aggregator earnings PDF layouts into a normalized earnings table, cleans and validates the driver-log sample, joins each driver-day to that date's weather and fuel price, and writes both the aggregator earnings table and the cleaned driver dataset as Parquet and CSV with a data dictionary and per-source lineage. The whole pipeline re-runs from one documented command, and a short notebook validates data quality and shows day-of-week and monthly earnings patterns.

1. A Python aggregator PDF parser covering Uber, Ola, Rapido, and Namma Yatri, logging unparseable rows.
2. A cleaned, enriched, de-identified driver dataset (Parquet and CSV) with schema, data dictionary, and per-source lineage.
3. An analytics notebook with data-quality validation and day-of-week and monthly earnings patterns, plus a README and a one-command re-run.

About the Client / Brand

BARDWS is a 4,200-member Karnataka Society founded in 2008 in Wilson Garden, governed by an elected executive committee and funded by member dues and a state welfare grant. Its guiding principle for member data is the tagline 'Your earnings, your data': member data stays de-identified and under the Sangha's control. The Sangha supplies the sample datasets and aggregator statement PDFs for this engagement.

Brand tone: trustworthy, plain-spoken, member-first.

Requirements

Build a reproducible Python pipeline with three parts: an aggregator PDF parser, a cleaning and join step, and a validation notebook. Every requirement below is testable.

Aggregator PDF Parser

- Parse the provided aggregator statement PDFs in assets/03_aggregator_pdfs/ for all four layouts: Uber, Ola, Rapido, and Namma Yatri (12 sample files, three per aggregator).
- Extract a normalized per-row earnings record with these fields: aggregator, statement_period_start, statement_period_end, txn_date, trips, rider_payment_inr, platform_fee_inr, net_earnings_inr.
- The layouts vary in columns, labels, and date format across years; the parser handles all four. Any row that cannot be parsed is written to parse_failures.log with file name and reason. No row is silently dropped.

Dataset and Joins

- Tidy the driver-log sample assets/02_driver_logs/member_driver_logs_sample.csv (200 drivers, Jan 2023 to Dec 2024, about 131,000 rows): remove the 80 fully duplicated driver-day rows; flag the 12 outlier rows where hours_worked equals 26; record the missing km_driven (about 1.5%) and missing gross_fare_inr (about 0.6%) in the data-quality report and impute or null per a documented rule.
- Enrich each driver-day by log_date: join the IMD weather sample on date, adding rainfall_mm, temp_max_c, temp_min_c, humidity_pct; join the fuel-price sample on date for IndianOil Petrol retail_price_inr_per_litre.
- Write the normalized aggregator earnings table and the cleaned driver dataset each as both Parquet and CSV.
- Deliver a documented schema, a data dictionary for every column, and per-source lineage (source system, ingest timestamp, transform version) for each output.

Analytics Notebook

- A Jupyter notebook with a data-quality validation section that reports the duplicate count, the missing-value rates, and the outlier rows.
- A day-of-week earnings analysis from gross_fare_inr.
- A monthly earnings analysis across the 24-month window.

Reproducibility and De-identification

- The pipeline re-runs end to end from one documented command and regenerates both outputs.
- Dependencies are pinned and any random step is seeded so a re-run reproduces the outputs.
- The dataset keeps only the supplied anonymised driver_id values and adds no field that could re-identify a driver.

Technical Requirements

- Language: Python 3.11+.
- Data libraries: pandas, numpy, pyarrow.
- Outputs: Parquet (primary) and CSV (interoperable) for both tables.
- PDF parsing: an open-source library (pdfplumber or PyMuPDF).
- Reproducibility: pinned dependency versions; a fixed random seed; a single documented run command.
- Code organization: one module per pipeline stage (parse, tidy, join, validate); no function longer than 60 lines.
- No network: the pipeline reads only local files in assets/ and writes only local outputs. No third-party or cloud calls.
- License: all delivered source under MIT or Apache 2.0.

Allowed Tech Stack

1. Python 3.11+ with pandas, numpy, pyarrow.
2. pdfplumber or PyMuPDF for PDF parsing.
3. Jupyter for the analytics notebook; Matplotlib or Plotly for figures.

Not Allowed

- Paid or closed APIs, or any service that sends member data off the local environment.
- Cloud storage or processing for the member data.
- Any field or join that re-identifies an individual driver.
- A dashboard, a deployed app, or a trained forecasting model in this milestone.

Reference Materials

References used this milestone are under assets/.

#	Asset	File / Path
1	Anonymised driver-log sample (200 drivers, 24 months)	assets/driver_logs/member_driver_logs_sample.csv
2	Aggregator earnings PDFs (four layouts, twelve files)	assets/aggregator_pdfs/
3	Bengaluru fuel-price sample 2018 to 2024	assets/fuel_prices/bengaluru_fuel_prices_2018_2024.csv
4	IMD Bengaluru weather sample 2018 to 2024	assets/weather/imd_bengaluru_weather_2018_2024.csv
5	Data dictionary for the sample sources	assets/DATA_DICTIONARY.md

How to use these references: the samples define the schemas and behaviours to support. The aggregator PDFs vary in layout, columns, and date format across years; the parser handles all four. The driver-log sample contains missing values, duplicate rows, and outliers; the cleaning step catches them. Join keys are the date columns named in the data dictionary.

Not used this milestone (do not implement): the brand kit (assets/01_brand_kit/), the fare-structure gazette (assets/05_fare_gazettes/), the ward shapefile and demand-zone maps (assets/06_ward_geo/), the UI string skeleton and fonts (assets/08_ui_strings/), the privacy and de-identification policy (assets/09_privacy_policy/), the device matrix (assets/10_device_matrix/), the wireframes (assets/11_wireframes/), and the ISEC and IISc data-handling agreement (assets/12_data_handling_agreement/). These belong to later milestones.

Deliverables

#	Item	Format	Notes
D-01	Aggregator PDF parser	.py	Handles all four layouts; logs parse failures
D-02	Normalized aggregator earnings table	.parquet and .csv	With schema and per-source lineage
D-03	Cleaned, enriched driver dataset	.parquet and .csv	De-identified; with data dictionary and per-source lineage
D-04	Analytics notebook	.ipynb	Data-quality validation; day-of-week and monthly patterns
D-05	README	.md	Setup and the one-command pipeline run

File Naming Convention

```
bardws_[component]_[detail]_v[N]_[YYYY-MM-DD].[ext]
```

Example: bardws_dataset_driver-joined_v1_2026-06-12.parquet

- Lowercase kebab-case for component and detail.
- No spaces. No alternate naming structures.
- Version every file with v[N].

Folder Structure

```
DA-009/
  01_parser/          # D-01
  02_datasets/        # D-02, D-03 (parquet, csv, schema, data dictionary, lineage)
  03_notebook/        # D-04
  assets/             # provided inputs, unmodified
  LICENSE             # MIT or Apache 2.0
  README.md           # D-05
```

Build / Export Specifications

- Run: one documented command regenerates D-02 and D-03 from the assets.

- Source: complete and reproducible from a fresh clone; pinned dependencies; seeded.
- Outputs: both tables exist as Parquet and CSV at the documented paths.

Scope Boundaries

DO

- Parse all four aggregator layouts across the provided samples.
- Tidy the driver-log sample and enrich it with weather and fuel by date.
- Write both tables as Parquet and CSV with schema, data dictionary, and lineage.
- Keep the dataset de-identified and the pipeline reproducible from one command.

DO NOT

- Build a dashboard, audience views, or any deployed app.
- Train a forecasting model or run a commission-impact study.
- Add encryption at rest, per-view scoping, or k-anonymity suppression.
- Add Kannada or English UI localization, fonts, or an offline PDF digest.
- Parse the fare gazette, join the ward shapefile, or run device-matrix QA.
- Send member data to any third-party or cloud service.

Tool Requirements

- Required tools: Python 3.11+, pandas, numpy, pyarrow, an open-source PDF parser, Jupyter.
- Acceptable alternatives: Matplotlib or Plotly for notebook figures.
- Forbidden: paid services that transmit member data externally; proprietary libraries that prevent reproducibility or handover.

Acceptance Checklist

1. The aggregator parser extracts a normalized earnings table from the Uber, Ola, Rapido, and Namma Yatri samples.
2. Unparseable rows are logged rather than dropped.
3. The normalized aggregator earnings table is delivered as Parquet and as CSV.
4. The cleaned driver dataset is delivered as Parquet and as CSV.
5. The 80 duplicate driver-day rows are removed and the 12 outlier rows are flagged.
6. Each driver-day is joined to that date's weather and fuel price.
7. A data dictionary and per-source lineage are delivered for the outputs.
8. The dataset adds no field that could re-identify a driver.
9. The notebook contains a data-quality validation section and day-of-week and monthly analyses.
10. The pipeline re-runs from one documented command, with pinned dependencies and a fixed seed.

Final Goal

When this is done, BARDWS has one cleaned, documented dataset built from its own sample sources: the four aggregators' earnings parsed into a single table, every driver-day joined to that day's weather and fuel price, every column defined in a data dictionary, and every value traceable back to its source. The whole thing regenerates from one command, keeps only anonymised driver IDs, and gives the Sangha the trustworthy data foundation it needs before the dashboard and the forecasts are built in later milestones.